

Task 1

Integrantes

- Sergio Orellana 221122
- Rodrigo Mansilla 22611
- Ricardo Chuy 221007

Escenario 1

Si yo construyera una red secuencial clasica de 150 capas, no asumiria que más profundidad siempre mejora el resultado. En una arquitectura plana tan profunda, el entrenamiento tiende a deteriorarse por dos problemas: desvanecimiento del gradiente y degradacion.

El desvanecimiento del gradiente ocurre porque, durante backpropagation, el gradiente se propaga a traves de muchas capas y su magnitud puede hacerse demasiado pequeña. De forma intuitiva, si el gradiente efectivo se comporta como $g = g_1 * g_2 * \dots * g_L$ y muchos terminos tienen magnitud menor que 1, entonces el producto se acerca a 0. En consecuencia, las primeras capas casi dejan de aprender y la red no ajusta bien filtros basicos como bordes, manchas y texturas.

Ademas, aunque yo use tecnicas modernas como Batch Normalization, seguiria enfrentando el problema de degradacion. Este fenomeno significa que al aumentar demasiado la profundidad, incluso el error de entrenamiento puede empeorar respecto a una red más corta. Por tanto, el problema no es solo de generalizacion, sino de optimizacion. En una startup, eso se traduce en más tiempo de prueba, menor estabilidad y más riesgo tecnico.

Por eso yo justifico el uso de ResNet. En un bloque residual, la salida se expresa como $y = F(x) + x$. Esa conexion de atajo crea un camino identidad que facilita el flujo del gradiente, porque la derivada local pasa a ser $dy/dx = dF(x)/dx + 1$. Asi, aunque $dF(x)/dx$ sea pequeno, la señal no desaparece por completo. Ademas, si una capa adicional no aporta valor, el bloque puede aprender algo cercano a $F(x) = 0$ y conservar la entrada. En consecuencia, ResNet vuelve viable entrenar redes muy profundas sin colapsar, lo cual si tiene sentido para un proyecto real.

Escenario 2

En este problema biologico no espero una sola escala visual. La antracnosis puede aparecer como puntos oscuros pequeños, mientras que el moho polvoriento puede cubrir regiones extensas de la hoja. Por eso considero que Inception es una arquitectura adecuada: procesa la misma entrada en ramas paralelas con filtros de distinto tamaño, por ejemplo 3×3 y 5×5 .

Desde mi perspectiva, esa topología en paralelo me permite capturar patrones finos y patrones amplios al mismo tiempo. El filtro 3×3 responde mejor a texturas locales y pequeños detalles; en cambio, el filtro 5×5 ve un campo receptivo mayor y ayuda a modelar estructuras extendidas. Luego, al concatenar ambas ramas, obtengo una representación más rica y más coherente con la heterogeneidad real de las enfermedades del mango.

Sin embargo, ejecutar muchas convoluciones grandes en paralelo puede elevar demasiado el costo. Si yo aplicara directamente una convolución 5×5 sobre una entrada con C_{in} canales para obtener C_{out} canales, el costo dominante sería proporcional a $25 * C_{in} * C_{out} * H * W$. Si antes inserto una convolución 1×1 para reducir canales de C_{in} a C_r , entonces el costo pasa aproximadamente a $C_{in} * C_r * H * W + 25 * C_r * C_{out} * H * W$, con $C_r < C_{in}$.

Esa reducción es clave para el negocio. Menos canales intermedios implican menos memoria, menos operaciones y menos tiempo de GPU en AWS o Google Cloud. Por tanto, yo uso las convoluciones 1×1 como cuello de botella inteligente: mantengo la ventaja multi-escala de Inception, pero evito que el experimento se vuelva caro e insostenible para una startup pequeña.

Escenario 3

Si el modelo se va a ejecutar en un teléfono Android de gama baja y sin internet, yo debo priorizar eficiencia, tamaño y latencia. Por eso MobileNet me parece una opción muy razonable. Su idea central es la Depthwise Separable Convolution, que divide la convolución estándar en dos pasos.

Primero, la depthwise convolution aplica un filtro espacial por cada canal de entrada. Después, la pointwise convolution, que es una convolución 1×1 , mezcla la información entre canales. En otras palabras, separo el trabajo en dos partes: filtrado espacial y combinación de canales.

La ventaja computacional es fuerte. Una convolución estándar con kernel $D_k \times D_k$, M canales de entrada y N de salida requiere un costo proporcional a $D_k^2 * M * N$. En cambio, una depthwise separable convolution reduce ese costo a $D_k^2 * M + M * N$. Como normalmente $D_k^2 * M * N$ es mucho mayor que $D_k^2 * M + M * N$, el ahorro en parámetros y operaciones es significativo.

Aun así, si pago un costo técnico. Al separar ambos pasos, sacrifico parte de la expresividad de una convolución estándar, porque ya no aprendo de forma completamente conjunta las interacciones espaciales y entre canales en una sola operación.

Como director del proyecto, acepto ese costo porque el contexto comercial lo exige. El agricultor necesita una app pequeña, rápida y confiable en un dispositivo con recursos limitados y sin conectividad. En ese escenario, un modelo ligeramente menos expresivo pero

desplegable offline aporta más valor real que un modelo más pesado que no pueda ejecutarse bien en producción. Mi criterio no es maximizar accuracy aislada, sino maximizar utilidad práctica en el campo.

Referencias

- GeeksforGeeks. (2023, April 18). ML | Inception Network V1. GeeksforGeeks. <https://www.geeksforgeeks.org/machine-learning/ml-inception-network-v1/>
- GeeksforGeeks. (2025a, June 30). Depth wise Separable Convolutional Neural Networks. GeeksforGeeks. <https://www.geeksforgeeks.org/machine-learning/depth-wise-separable-convolutional-neural-networks/>
- GeeksforGeeks. (2025b, July 15). Understanding GoogLeNet Model CNN Architecture. GeeksforGeeks. <https://www.geeksforgeeks.org/machine-learning/understanding-googlenet-model-cnn-architecture/>
- GeeksforGeeks. (2025c, July 23). Mobilenet V2 Architecture in Computer Vision. GeeksforGeeks. <https://www.geeksforgeeks.org/computer-vision/mobilenet-v2-architecture-in-computer-vision/>
- GeeksforGeeks. (2026a, January 7). Residual Networks (ResNet) Deep learning. GeeksforGeeks. <https://www.geeksforgeeks.org/deep-learning/residual-networks-resnet-deep-learning/>
- GeeksforGeeks. (2026b, January 15). Vanishing and exploding gradients problems in deep learning. GeeksforGeeks. <https://www.geeksforgeeks.org/deep-learning/vanishing-and-exploding-gradients-problems-in-deep-learning/>